1. Sequential 3D Raytracer Design

1.1 Overview

The application being parallelized is a 3D raytracer implemented in C#, based on Peter Shirley's "Ray Tracing in One Weekend" tutorial. A raytracer is a rendering technique that simulates the physics of light to generate photorealistic images by tracing the path of light rays as they interact with objects in a 3D scene.

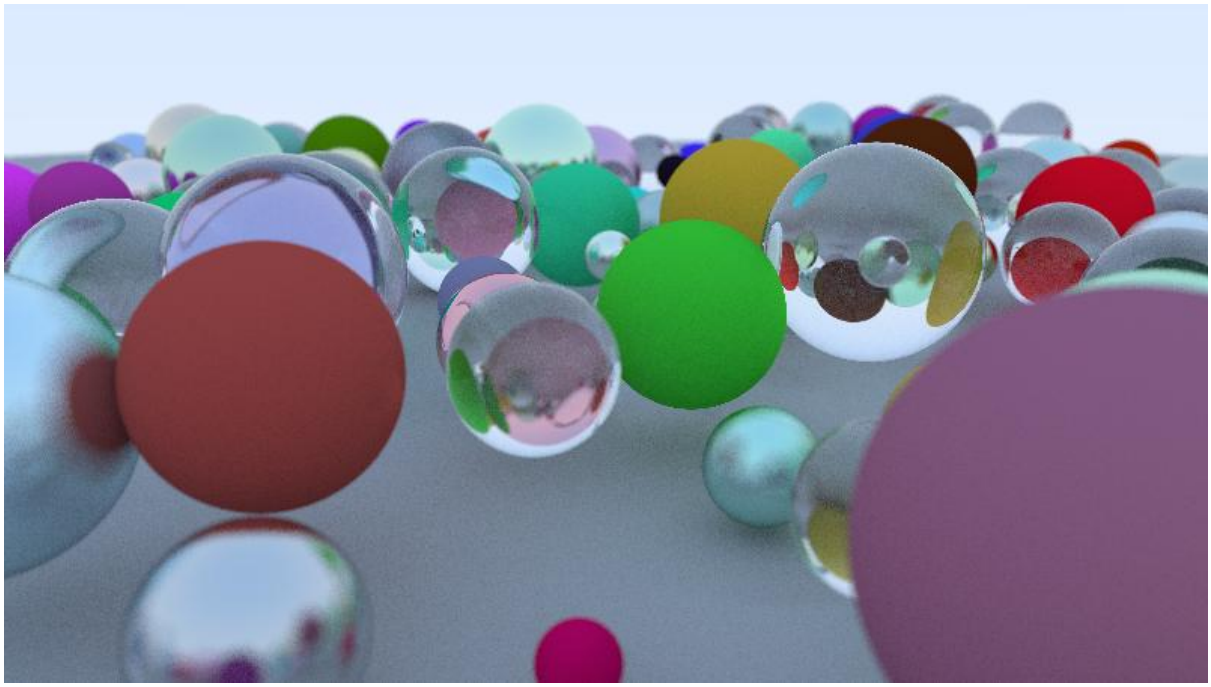

Figure 1: Rendered 3D Scene using the sequential raytracer

From a user perspective, the raytracer takes scene configuration parameters (such as camera position, field of view, image resolution) and generates a 3D world containing spheres of random sizes and material types (diffuse, metal, and glass). The program simulates the lighting for each pixel of the image and generates an image file in the PPM format with the resulting rendered 3D environment.

The application can process three material types:

- Diffuse: Rays bounce off in random directions, simulating a matte surface.
- Metal: Rays reflect directly, with reflection strength controlled by roughness.
- Dielectric (Glass): Rays are refracted based on an index of refraction (IOR) value.

1.2 Software Architecture

The application follows an object-oriented design consisting of a standard graphics pipeline using the following subsystems:

- Vector and geometric operations
- Scene objects and hit detection
- Material/light interaction models (Lambertian, Metal, Dielectric)
- Camera and ray generation
- RNG, colour conversion, mathematical constants

1.2.1 Vec3, Point3 and Color

The Vec3 class is a fundamental 3D vector class with x, y, z components that implements mathematical functions like addition, subtraction, multiplication, and division, in addition to the geometric operations of dot product, cross product, and normalize. Utility methods for random vector generation are also included. Point3 and Color inherit from Vec3 and exist for semantic clarity.

1.2.2 Ray

Represents a light ray with an origin point and direction vector using the formula

$$\mathbf{P(t) = Origin + t * Direction}$$

The At(t) method is included to compute points along the rays.

1.2.3 IHittable and HittableList

IHittable is a parent interface for objects that can be intersected by rays (Spheres). The HittableList class is used as a container for the collection of hittable objects.

1.2.4 HitRecord

Data structure storing intersection information containing the hit point, surface normal, ray parameter, and material reference. Contains a method to determine if a ray hit the front or back face of the hittable object.

1.2.5 Sphere

An implementation of IHittable for spheres storing a center point, radius, and material reference, using the quadratic formula to identify ray-sphere intersections.

1.2.6 Material

An Abstract Material base class with a Scatter method used by the following material systems:

- Lambertian: Diffuse reflection with random scatter direction
- Metal: Specular reflection
- Dielectric: Refraction and reflection

1.2.7 Camera

The camera is the core rendering engine and contains the main rendering loop using the specified aspect ratio, image dimensions, FOV, camera positioning, and depth of field settings.

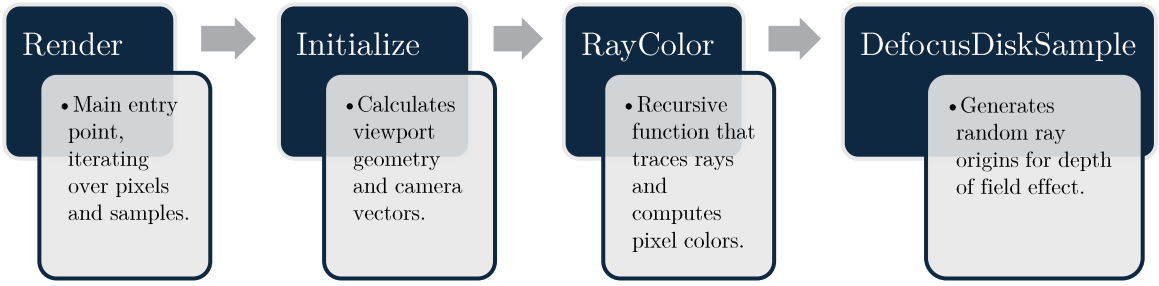

Figure 2: Camera class pipeline

1.2.8 Utility

Static helper class for random number generation (RNG), mathematical constants (pi, infinity) and conversions (degrees to radians).

1.2.9 Program

Application entry point that generates the randomized 3D scene before each render begins.

1.3 Rendering Algorithm

The sequential rendering algorithm follows these steps:

1. World Initialization
    a. The 3D world is initialized, generating a very large sphere to represent the ground plane and many smaller spheres of random sizes, materials and positions.
2. Camera Initialization
    a. The camera calculates the viewport dimensions based on FOV and aspect ratio.
    b. Camera vectors (u, v, w), pixel grid coordinates, and depth of field defocus disks are computed.

3. Hit Detection
    a. Iterate through all objects in HittableList and test for intersection.
    b. Track the closest hit and return hit information (point, normal, material)
4. Per-Pixel Rendering Loop
    a. Uses nested loops to traverse each pixel on the image.
    b. Generates a ray from the camera through the pixel with a random offset
    c. Compute the RayColor
        i. If depth = 0, the ray has bounced too many times, so return black.
        ii. If the material scatters the ray, recursively call RayColor with the scattered ray and reduced depth.
        iii. If the material absorbs the ray, return black.
        iv. If the ray hits nothing, return the sky colour using a basic gradient.
5. Generate PPM Image File from each pixel's simulation.

2. Identifying Potential Parallelism

2.1 Profiling Analysis

Before identifying any parallelization opportunities, I performed a performance profile on the sequential application (400 by 225 pixel, 20 samples per pixel, max depth 20). The profiling results revealed computational hotspots across function calls.

97.75% of execution time was spent in the Camera.Render() method, with 97.15% specifically in the recursive RayColor() function. 25.50% was spent testing ray-object intersections in HittableList.Hit().

This profiling data confirms that the rendering loop is the overwhelming bottleneck and primary target for parallelization. The high percentage in RayColor and HittableList.Hit indicates that both ray tracing logic and intersection testing are computationally intensive.

The RayColor's smaller Self CPU % indicates that each ray is bouncing multiple times throughout the scene, with the recursion overhead being minimal compared to the intersection testing. Additionally, the recursive structure naturally limits parallelism due to sequential dependency.

2.2 Identification of Independent Loops

The Camera.Render() method contains three nested loops and one recursive loop that represent the primary computational workload.

Each pixel in the x & y loops is computed individually and does not depend on other pixels to successfully compute, making these two loops a valid candidate for parallelism.

2.3 Identification of Data Dependency

The following data structures are read by multiple rays but never modified during rendering:

- HittableList
- Spheres
- Materials
- Camera
- Viewport Geometry

These read-only structures enable parallel computation with no synchronization required for reading.

2.4 Identification of Synchronization Requirements

The SteamWriter is a sequential process and requires buffering the pixels in memory with sequential writing after the parallel computation.

The random number generator is not safe for concurrent calls as it uses a single shared static Random instance in the Utility class. When multiple threads call RandomDouble() concurrently, a race condition is created as each call to NextDouble() occurs. To resolve this, each thread requires its own isolated Random instance.

2.5 Parallelization Strategy

After analysing the loop structure and dependencies, the .NET Task Parallel Library, was selected as the primary parallelization approach. This approach maps naturally to the independent pixel computations in the outer loop, where each iteration represents a row of pixels with no dependencies between the rows.

3. Implementation Details

3.1 Computation Mapping

The parallelization employs a data parallel approach using row-level decomposition. The rows of the output image (equal to the image width) are distributed across available processor cores as independent tasks, with each row containing a column of pixels (equal to the image height) requiring a specified number of samples each (20 by default). This decomposition provides sufficient granularity while maintaining simple implementation and good cache locality.

The Task Parallel Library automatically handles the mapping of rows to processor cores through its work-stealing scheduler. Each core of the system has worker threads continuously pull row tasks from a shared work queue, with idle threads stealing work from busy threads to maintain load balance.

3.2 Synchronization

The computation mapping separates read-only scene data (replicated across all threads) from write-only pixel data (partitioned by row), eliminating the need for synchronization during computation.

3.3 Code Modifications

In terms of the rendering process, only two classes required modification.

3.3.1 Utility.cs

The random number generation now uses the [ThreadStatic] attribute to create per-thread Random instances, eliminating race conditions on the shared Random object in the Utility class.

```
// Thread-local Random instance to avoid race conditions in parallel execution
[ThreadStatic]
3 references
private static Random? random;

1 reference
private static Random GetRandom()
{
    if (random == null)
    {
        int seed = Environment.CurrentManagedThreadId ^ (int)DateTime.Now.Ticks;
        random = new Random(seed);
    }
    return random;
}
```

Figure 3: Thread-local GetRandom() method of the Utility class

3.3.2 Camera.cs

A 2D Colour array was added as a buffer in memory to allow computed pixels to be written to the final image in parallel. The outer loop responsible for the columns was replaced with 'Parallel.For' as the main parallelisation implementation. The addition of parallel pixel buffering required the inclusion of a sequential loop to write the image buffer to the PPM file after the completed computation. The per-pixel computation logic and other classes required no modifications as their operations were naturally thread-safe.

```
public void Render(IHittable world, string outputFile, ParallelOptions parallelOptions)
{
    Initialize();

    // Image buffer to store pixels in any order
    Color[,] imageBuffer = new Color[ImageWidth, imageHeight];

    Parallel.For(0, imageHeight, parallelOptions, j =>
    {
        // Each thread process one row at a time
        for (int i = 0; i < ImageWidth; i++)
        {
            Color pixelColor = new Color(0, 0, 0);

            for (int sample = 0; sample < SamplesPerPixel; sample++)
            {
                Vec3 offset = new Vec3(0, 0, 0);
                Vec3 pixelCenter = pixel00Loc + ((i + offset.X()) * pixelDeltaU) + ((j + offset.Y()) * pixelDeltaV);

                Point3 rayOrigin = (DefocusAngle <= 0) ? center : DefocusDiskSample();
                Vec3 rayDirection = pixelCenter - rayOrigin;
                Ray r = new Ray(rayOrigin, rayDirection);

                Vec3 sampleColor = RayColor(r, MaxDepth, world);
                pixelColor = new Color(
                    pixelColor.X() + sampleColor.X(),
                    pixelColor.Y() + sampleColor.Y(),
                    pixelColor.Z() + sampleColor.Z()
                );
            }

            Vec3 finalColor = pixelColor / SamplesPerPixel;
            imageBuffer[i, j] = new Color(finalColor.X(), finalColor.Y(), finalColor.Z());
        }
    });

    // Write buffered pixels to PPM file sequentially
    using (StreamWriter writer = new StreamWriter(outputFile))
    {
        writer.WriteLine($"P3\n{ImageWidth} {imageHeight}\n255");

        for (int j = 0; j < imageHeight; j++)
        {
            for (int i = 0; i < ImageWidth; i++)
            {
                ColorUtility.WriteColor(writer, imageBuffer[i, j]);
            }
        }
    }
}
```

Figure 4: Parallelized Render() method of the Camera class

4. Performance Profiling

4.1 Test Configuration

4.1.1 Hardware Configuration:

- CPU Model: AMD Ryzen 5 7600X 6-Core Processor
- Base Clock: 4.7 GHz
- Physical Cores: 6
- Logical Processors (Virtual/SMT): 12
- OS: Windows 11
- .NET Runtime: .NET 8.0
- Compiler Settings: Release mode

4.1.2 Software Environment:

- Runtime: .NET 8.0
- Compiler: C# with Release build
- Parallelism: .NET Task Parallel Library (System.Threading.Tasks)

4.1.3 Render Configuration:

- Image Resolution: 400 × 225 pixels (90,000 pixels)
- Samples Per Pixel: 20
- Max Ray Depth: 20
- Field of View: 40°
- Focus Distance: 10
- Test Iterations: 10 runs per max degree of parallelism
    - None (Sequential), 1, 2, 4, 8, 12

4.2. Results Analysis

Max Degrees of Parallelism	None	1	2	4	8	12
Min	47.771	50.359	27.794	14.156	9.915	8.132
Max	66.527	57.475	39.246	17.956	14.295	9.781
Average	57.9159	53.7703	31.0768	15.4189	10.8095	8.8954

Figure 5: Min, Max and Average execution times across 10 tests

The execution time results demonstrate the effectiveness of parallel processing in reducing render time. The sequential baseline averaged 57.9 seconds with a high variance due to OS background processes. The parallel implementation achieved dramatic time reductions down to an average of 8.9s.

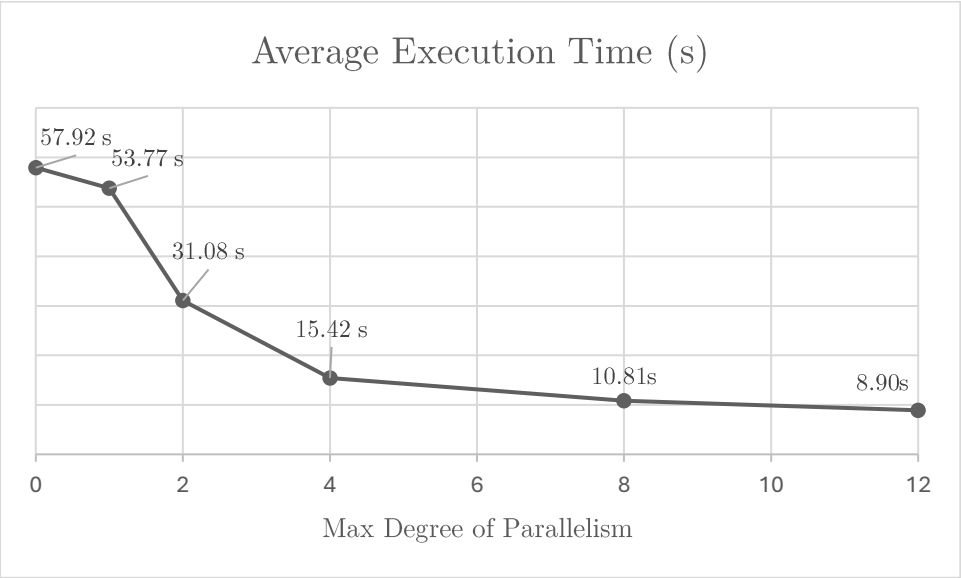

Figure 6: Graph of the Average Execution Time by the Max Degree of Parallelism

Figure 6 illustrates the inverse relationship between execution time and parallelism level. The graph shows a steep initial decline from sequential (57.9s) through 4 cores (15.4s), representing the most significant performance gains. The trend shows diminishing returns beyond 8 cores, with tests beyond 12 cores showing negligible change due to hardware limitations. Figure 7 shows a percentage speedup of 551.08% at 12 cores, a significant increase in comparison to the small number of code modification required.

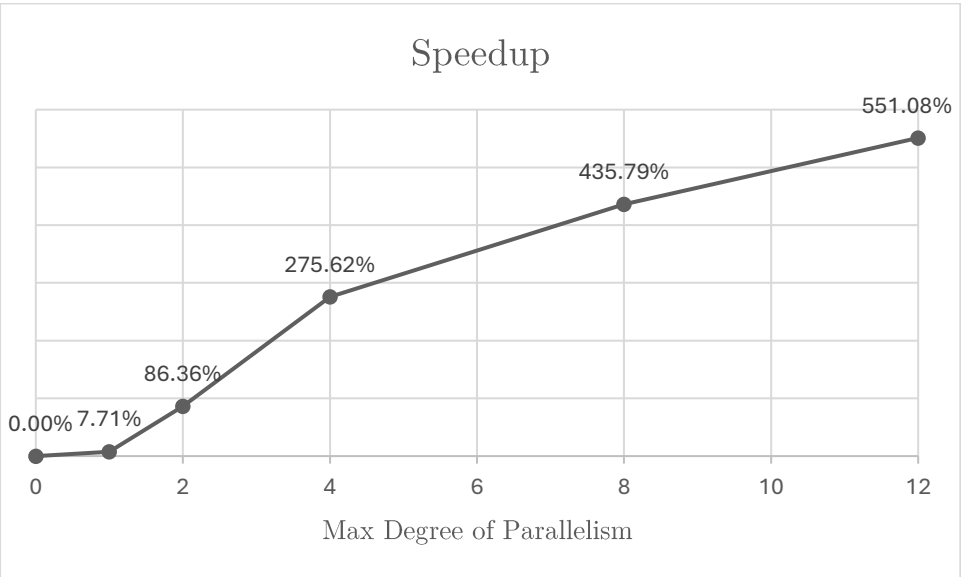

Figure 7: Graph of the Speedup Percentage by the Max Degree of Parallelism

4.3 Validity of Tests

Ensuring that the parallel implementation produced identical results to the sequential version was critical for test validation. Eliminating sources of non-determinism enabled direct pixel-by-pixel comparison between sequential and parallel outputs. The primary challenge in validating correctness was the inherent randomness in both scene generation and ray sampling.

```
static HittableList BuildWorld(int seed)
{
    Random random = new Random(seed);
    HittableList world = new HittableList();

    // Ground
    Material groundMaterial = new Lambertian(new Color(0.5, 0.5, 0.5));
    world.Add(new Sphere(new Point3(0, -1000, 0), 1000, groundMaterial));

    // Random small spheres
    int materialIndex = 0;
    for (int a = -8; a < 8; a++)
    {
        for (int b = -8; b < 8; b++)
        {
            double radius = random.NextDouble() * (1.2 - 0.25) + 0.25;
            Point3 center = new Point3(
                a * 2 + (random.NextDouble() * 0.2 - 0.1) * (1.2 - radius),
                radius + random.NextDouble(),
                b * 2 + (random.NextDouble() * 0.2 - 0.1) * (1.2 - radius)
            );
```

```
--- Render Settings ---
FOV: 40
Image Width: 400
Image Height: 225
Focus Distance: 10
Samples Per Pixel: 20
Max Depth: 20
Seed: 401

--- Output Settings ---
Number of Tests: 10
Max Degree Of Parallelism: 8
Show PPM File: Yes

1: Start Testing
2: Change Settings
3: Exit

Select option:
```

Figure 8: The modified BuildWorld() function and console settings.

To enable reproducible outputs, the BuildWorld() function of the Program class (in both sequential and parallelized versions) was modified to accept an optional seed parameter. A temporary fixed seed was also implemented into the Utility random object, which was later reverted after confirming there were no discrepancies between the versions.

5. Reflection and Conclusions

The parallelization effort was highly successful in significantly reducing render time through a parallel programming solution. The implementation achieved a 551.08% speedup at 12 cores, reducing average render time from 57.9 seconds to 8.9 seconds. The implementation maintained identical outputs to the sequential version when using fixed seeds for validation and the minimal code changes to only two classes demonstrated the effectiveness of parallelization.

The current implementation represents a well-optimized solution for the target hardware, though it is limited by the solution and language chosen. Flattening the pixels into a 1-dimensional array was considered but would've had a minimal improvement for a much larger code refactor. GPU acceleration through compute shaders or CUDA would provide the most dramatic improvement but would have required a large architectural redesign.

This project provided valuable experience with parallel programming concepts in the practical environment of 3D Graphics. Solving real-world parallel programming problems like the thread safety challenges provided an opportunity to learn how seemingly simple shared states can creates race conditions when accessed concurrently. Using high-level abstractions reinforced how modern parallel programming can leverage framework capabilities to significantly reduce development time while still achieving impressive performance increases.

Appendix

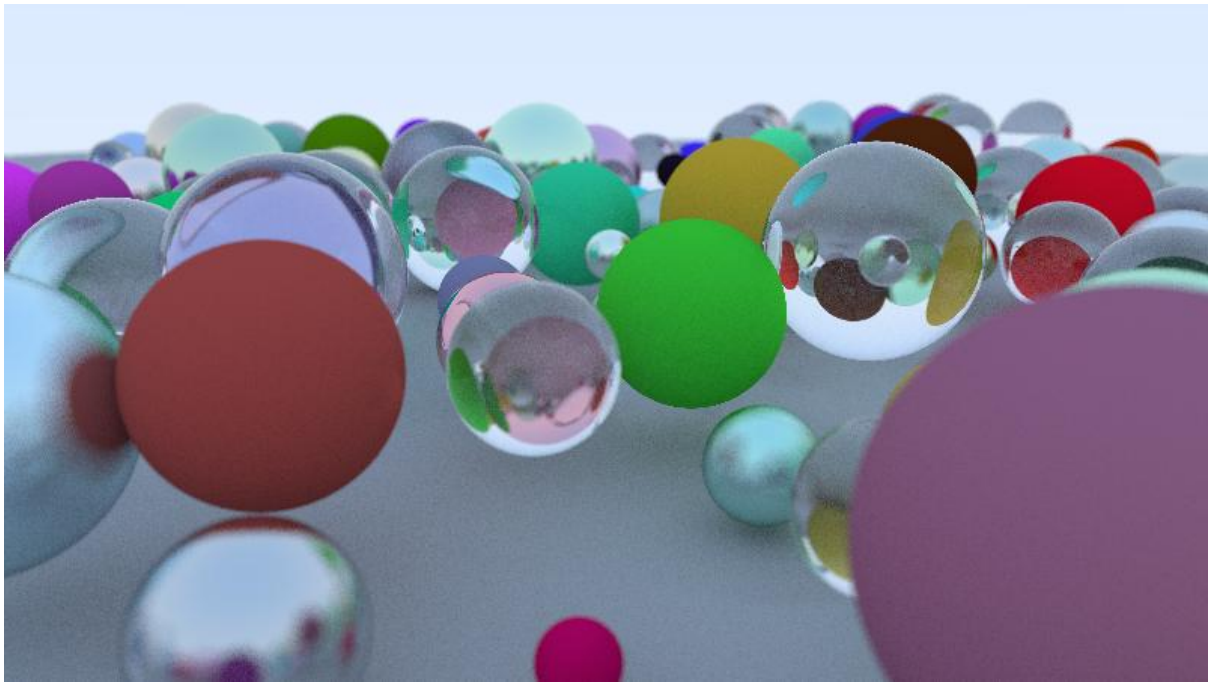

Figure 1: Rendered 3D Scene using the sequential raytracer

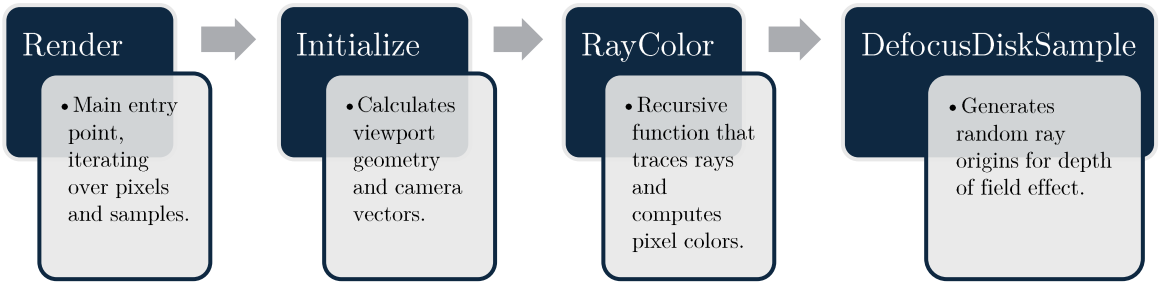

Figure 2: Camera class pipeline

```
// Thread-local Random instance to avoid race conditions in parallel execution
[ThreadStatic]
3 references
private static Random? random;

1 reference
private static Random GetRandom()
{
    if (random == null)
    {
        int seed = Environment.CurrentManagedThreadId ^ (int)DateTime.Now.Ticks;
        random = new Random(seed);
    }
    return random;
}
```

Figure 3: Thread-local GetRandom() method of the Utility class

```
public void Render(IHittable world, string outputFile, ParallelOptions parallelOptions)
{
    Initialize();

    // Image buffer to store pixels in any order
    Color[,] imageBuffer = new Color[ImageWidth, imageHeight];

    Parallel.For(0, imageHeight, parallelOptions, j =>
    {
        // Each thread process one row at a time
        for (int i = 0; i < ImageWidth; i++)
        {
            Color pixelColor = new Color(0, 0, 0);

            for (int sample = 0; sample < SamplesPerPixel; sample++)
            {
                Vec3 offset = new Vec3(0, 0, 0);
                Vec3 pixelCenter = pixel00Loc + ((i + offset.X()) * pixelDeltaU) + ((j + offset.Y()) * pixelDeltaV);

                Point3 rayOrigin = (DefocusAngle <= 0) ? center : DefocusDiskSample();
                Vec3 rayDirection = pixelCenter - rayOrigin;
                Ray r = new Ray(rayOrigin, rayDirection);

                Vec3 sampleColor = RayColor(r, MaxDepth, world);
                pixelColor = new Color(
                    pixelColor.X() + sampleColor.X(),
                    pixelColor.Y() + sampleColor.Y(),
                    pixelColor.Z() + sampleColor.Z()
                );
            }

            Vec3 finalColor = pixelColor / SamplesPerPixel;
            imageBuffer[i, j] = new Color(finalColor.X(), finalColor.Y(), finalColor.Z());
        }
    });

    // Write buffered pixels to PPM file sequentially
    using (StreamWriter writer = new StreamWriter(outputFile))
    {
        writer.WriteLine($"P3\n{ImageWidth} {imageHeight}\n255");

        for (int j = 0; j < imageHeight; j++)
        {
            for (int i = 0; i < ImageWidth; i++)
            {
                ColorUtility.WriteColor(writer, imageBuffer[i, j]);
            }
        }
    }
}
```

Figure 4: Parallelized Render() method of the Camera class

Max Degrees of Parallelism	None	1	2	4	8	12
Min	47.771	50.359	27.794	14.156	9.915	8.132
Max	66.527	57.475	39.246	17.956	14.295	9.781
Average	57.9159	53.7703	31.0768	15.4189	10.8095	8.8954

Figure 5: Min, Max and Average execution times across 10 tests

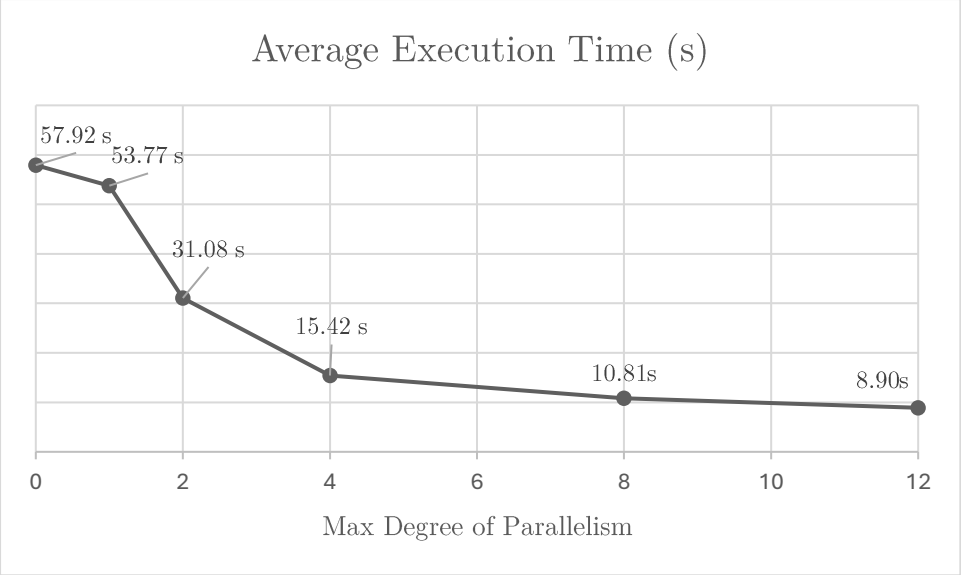

Figure 6: Graph of the Average Execution Time by the Max Degree of Parallelism

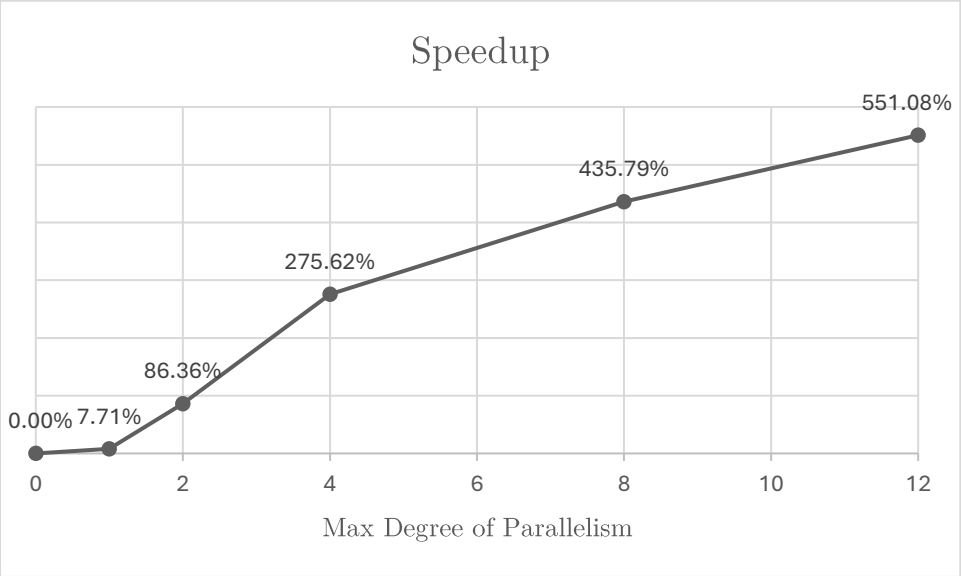

Figure 7: Graph of the Speedup Percentage by the Max Degree of Parallelism

```
static HittableList BuildWorld(int seed)
{
    Random random = new Random(seed);
    HittableList world = new HittableList();

    // Ground
    Material groundMaterial = new Lambertian(new Color(0.5, 0.5, 0.5));
    world.Add(new Sphere(new Point3(0, -1000, 0), 1000, groundMaterial));

    // Random small spheres
    int materialIndex = 0;
    for (int a = -8; a < 8; a++)
    {
        for (int b = -8; b < 8; b++)
        {
            double radius = random.NextDouble() * (1.2 - 0.25) + 0.25;
            Point3 center = new Point3(
                a * 2 + (random.NextDouble() * 0.2 - 0.1) * (1.2 - radius),
                radius + random.NextDouble(),
                b * 2 + (random.NextDouble() * 0.2 - 0.1) * (1.2 - radius)
            );
```

```
--- Render Settings ---
FOV: 40
Image Width: 400
Image Height: 225
Focus Distance: 10
Samples Per Pixel: 20
Max Depth: 20
Seed: 401

--- Output Settings ---
Number of Tests: 10
Max Degree Of Parallelism: 8
Show PPM File: Yes

1: Start Testing
2: Change Settings
3: Exit

Select option:
```

Figure 8: The modified BuildWorld() function and console settings

Max Degrees of Parallelism	None	1	2	4	8	12
Test 1	58.392	52.76	29.797	15.6	10.552	8.989
Test 2	47.771	53.359	31.321	14.945	10.181	8.738
Test 3	61.328	54.773	39.246	14.918	14.295	8.562
Test 4	66.527	57.475	30.923	14.2	10.299	8.854
Test 5	54.385	50.359	28.43	14.156	10.929	9.781
Test 6	53.955	54.315	30.709	17.956	10.315	8.132
Test 7	52.781	55.182	27.794	16.372	10.864	9.438
Test 8	61.571	56.57	29.795	15.503	10.548	8.88
Test 9	58.916	50.595	30.493	15.872	10.197	9.067
Test 10	63.533	52.315	32.26	14.667	9.915	8.513

Figure 9: Individual execution times across 10 tests

Function Name	Total CPU [unit, %]	Self CPU [unit, %]	Module
Raytracer (PID: 47020)	61059 (100.00%)	175 (0.29%)	Multiple modules
Raytracer.Program.Main(System.String[])	59840 (98.00%)	6 (0.01%)	raytracer
Raytracer.Program.RunTests()	59821 (97.97%)	4 (0.01%)	raytracer
Raytracer.Camera.Render(Raytracer.IH...	59683 (97.75%)	103 (0.17%)	raytracer
Raytracer.Camera.RayColor(Raytrac...	59316 (97.15%)	101 (0.17%)	raytracer
Raytracer.Camera.RayColor(Raytr...	43439 (71.14%)	107 (0.18%)	raytracer
Raytracer.HittableList.Hit(Raytrac...	15572 (25.50%)	4739 (7.76%)	raytracer
Raytracer.Dielectric.Scatter(Raytracer...	69 (0.11%)	34 (0.06%)	raytracer
Raytracer.Lambertian.Scatter(Raytrac...	66 (0.11%)	13 (0.02%)	raytracer
Raytracer.Metal.Scatter(Raytracer.Ra...	48 (0.08%)	7 (0.01%)	raytracer
Raytracer.Vec3.ctor(float64, float64, f...	9 (0.01%)	9 (0.01%)	raytracer
Raytracer.Vec3.op_Addition(Raytrace...	7 (0.01%)	7 (0.01%)	raytracer
Raytracer.Vec3.op_Multiply(float64, ...	3 (0.00%)	3 (0.00%)	raytracer
Raytracer.HitRecord.ctor()	2 (0.00%)	2 (0.00%)	raytracer
Raytracer.Vec3.op_Multiply(float64, Ray...	88 (0.14%)	88 (0.14%)	raytracer
Raytracer.Vec3.op_Addition(Raytracer.V...	79 (0.13%)	78 (0.13%)	raytracer
Raytracer.Vec3.RandomInUnitDisk()	41 (0.07%)	37 (0.06%)	raytracer
Raytracer.Vec3.op_Subtraction(Raytrac...	29 (0.05%)	29 (0.05%)	raytracer
Raytracer.ColorUtility.WriteColor(Syste...	24 (0.04%)	3 (0.00%)	raytracer
Raytracer.Camera.DefocusDiskSample()	1 (0.00%)	0 (0.00%)	raytracer
Raytracer.Camera.Initialize()	1 (0.00%)	1 (0.00%)	raytracer
[External Call] system.private.corelib.dll...	1 (0.00%)	1 (0.00%)	system.private.cor...
Raytracer.PpmViewer.Show(string)	120 (0.20%)	0 (0.00%)	raytracer
Raytracer.Program.SaveResultsToCsv(Syst...	6 (0.01%)	0 (0.00%)	raytracer
Raytracer.Program.DisplaySummary(Syst...	4 (0.01%)	2 (0.00%)	raytracer
Raytracer.Program.BuildWorld()	2 (0.00%)	2 (0.00%)	raytracer
Raytracer.Program.GetCurrentSettings()	1 (0.00%)	1 (0.00%)	raytracer
[External Call] system.private.corelib.dll!0...	1 (0.00%)	1 (0.00%)	system.private.cor...
Raytracer.Program.DisplayMenu()	12 (0.02%)	0 (0.00%)	raytracer
[External Call] system.console.dll!0x00007ff...	1 (0.00%)	1 (0.00%)	system.console
[Unwalkable]	1015 (1.66%)	1015 (1.66%)	
[External Call] system.private.corelib.dll!0x000...	24 (0.04%)	24 (0.04%)	system.private.cor...
[External Call] dynamicClass.IL_STUB_ReverseP...	3 (0.00%)	3 (0.00%)	system.windows.f...
[External Call] system.private.corelib.dll!0x000...	1 (0.00%)	1 (0.00%)	system.private.cor...
[External Call] system.windows.forms.dll!0x00...	1 (0.00%)	1 (0.00%)	system.windows.f...

Figure 10: Call Tree from sequential raytracer performance profile